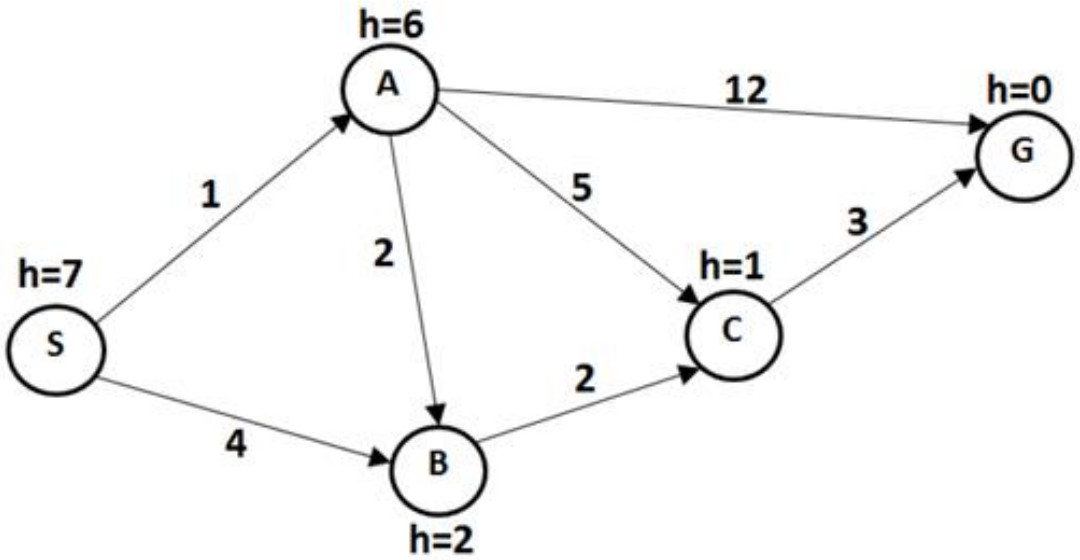
Artificial Intelligence
AI-2002
Lecture 6

Mahzaib Younas

Lecture Department of Computer Science

FAST NUCES CFD

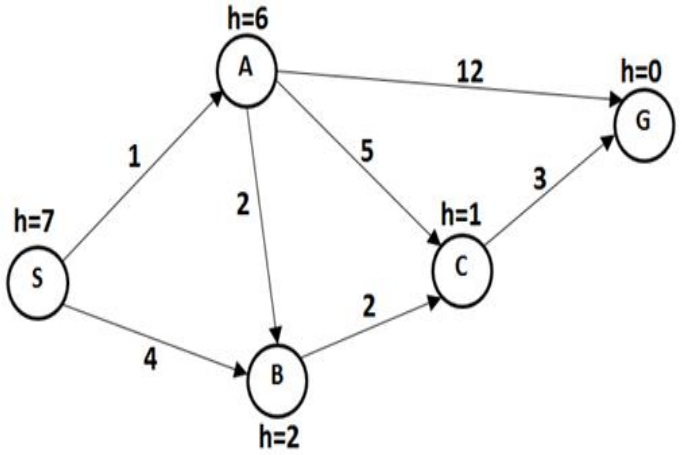
Check the admissibility and consistency of the following?



If heuristic is consistent the it must be admissible.
If heuristic is admissible then it may or may not be consistent.

Admissibility

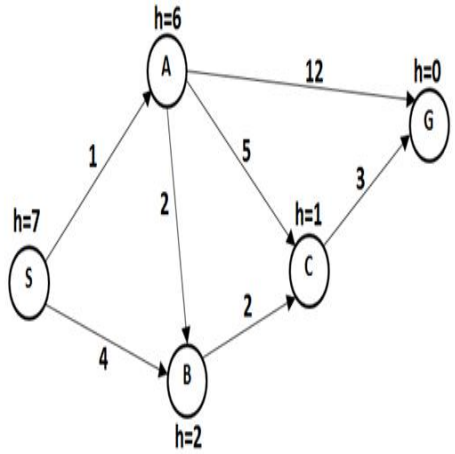
	Nodes	Admissibility h(n) ≤ f(n)
1	S	7 ≤ 13 7 ≤ 9 7 ≤ 9
2	A	6 ≤ 12 6 ≤ 7 6 ≤ 8
3	B	2 ≤ 5
4	C	1 ≤ 3



Consistency

$$h(n) - h(n`) \leq c(n, a, n`)$$

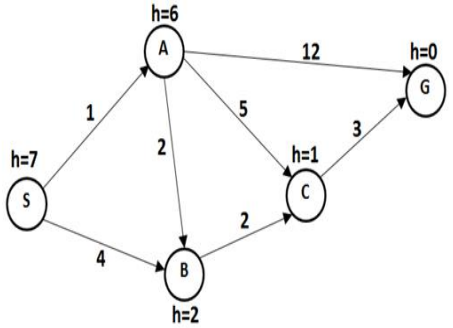
	Edges	Consistency		
1	S – A	7 – 6 ≤ 1 1 ≤ 1		
2	S – B	7 – 2 ≤ 4 5 ≤ 4	7 – 4 ≤ h(B) H(B) ≤ 3	7 – 4 ≤ 4 3 ≤ 4
3	A – B	6 – 2 ≤ 2 4 ≤ 2	h(b) ≤ 6 -2 h(b) ≤ 4	6 - 4 ≤ 2 2 ≤ 2
4	A – C	6 – 1 ≤ 5 5 ≤ 5		
5	A – G	6 – 0 ≤ 12 6 ≤ 12		
6	B – C	2 – 1 ≤ 2 1 ≤ 2	4 – 1 ≤ 2 3 ≤ 2	h(c) ≤ 4 – 2 h(c) ≤ 2
7	C – G	1 – 0 ≤ 3 1 ≤ 3		



Consistency

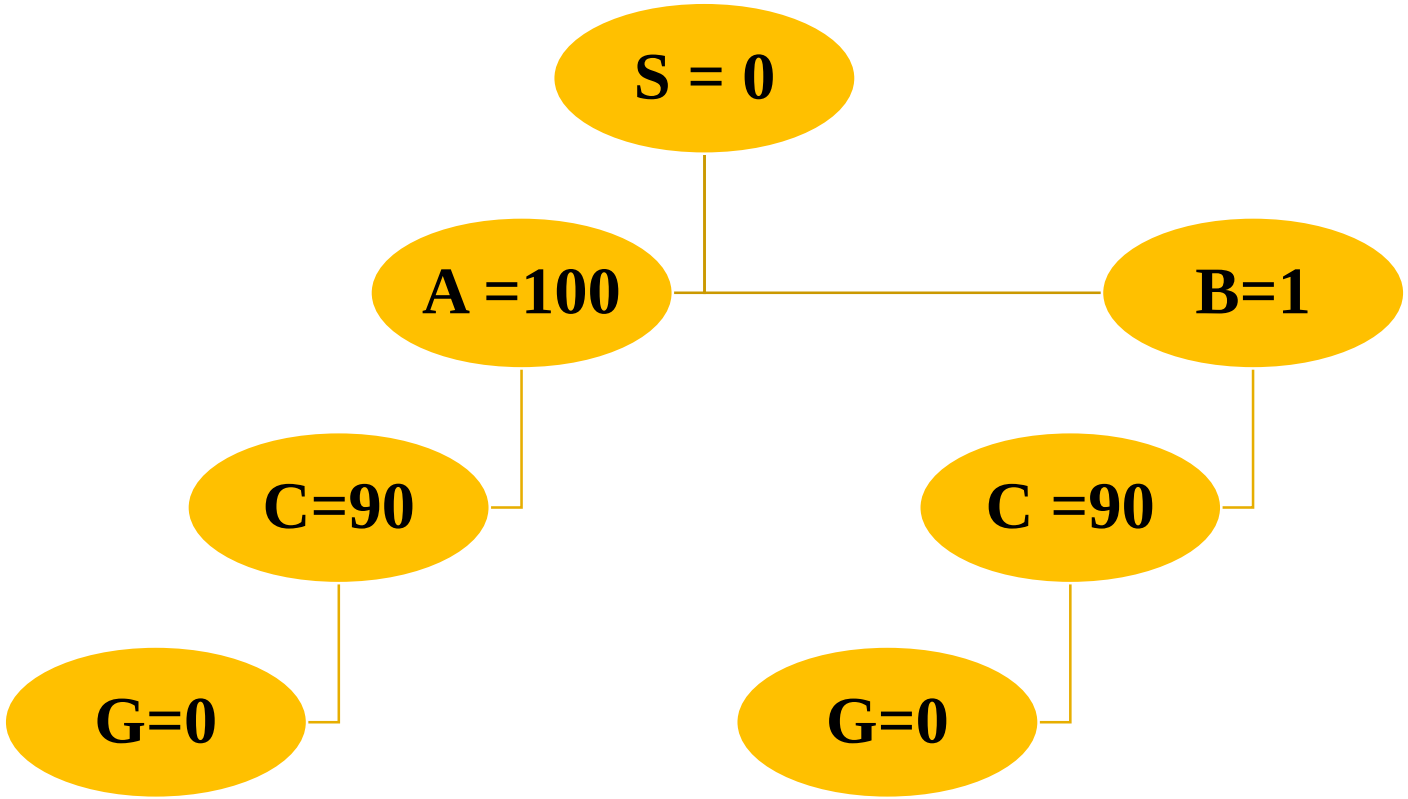
$$h(n) - h(n`) \leq c(n, a, n`)$$

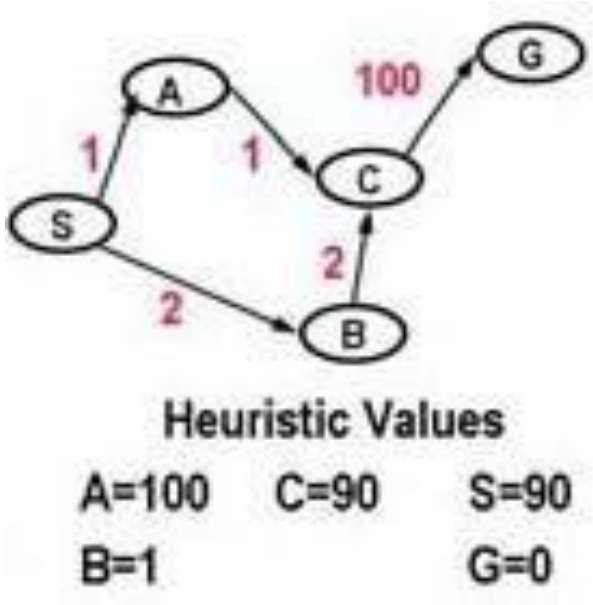
	Edges	Consistency		
1	S – A	7 – 6 ≤ 1 1 ≤ 1		
2	S – B	7 – 2 ≤ 4 5 ≤ 4	7 – 4 ≤ h(B) H(B) ≤ 3	7 – 4 ≤ 4 3 ≤ 4
3	A – B	6 – 2 ≤ 2 4 ≤ 2	h(b) ≤ 6 -2 h(b) ≤ 4	6 - 4 ≤ 2 2 ≤ 2
4	A – C	6 – 1 ≤ 5 5 ≤ 5		
5	A – G	6 – 0 ≤ 12 6 ≤ 12		
6	B – C	2 – 1 ≤ 2 1 ≤ 2	4 – 1 ≤ 2 3 ≤ 2	h(c) ≤ 4 – 2 h(c) ≤ 2
7	C – G	1 – 0 ≤ 3 1 ≤ 3		



Updated Heuristics values
H(c) = 2
H(b) = 4

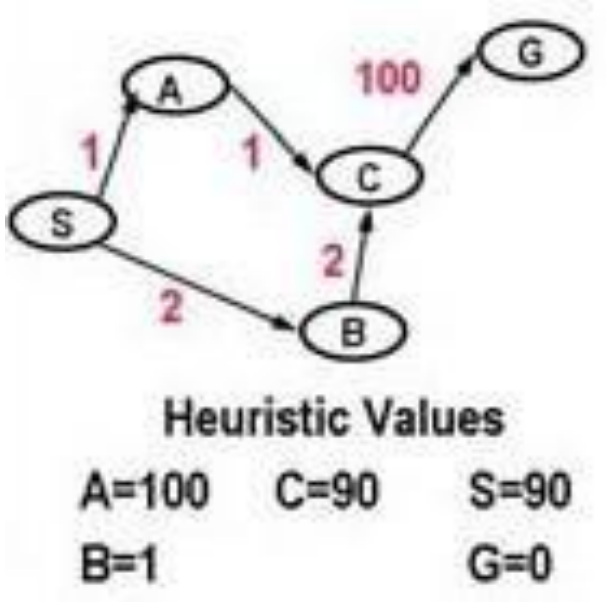
A* Algorithm without expanded list





A* without Expanded List

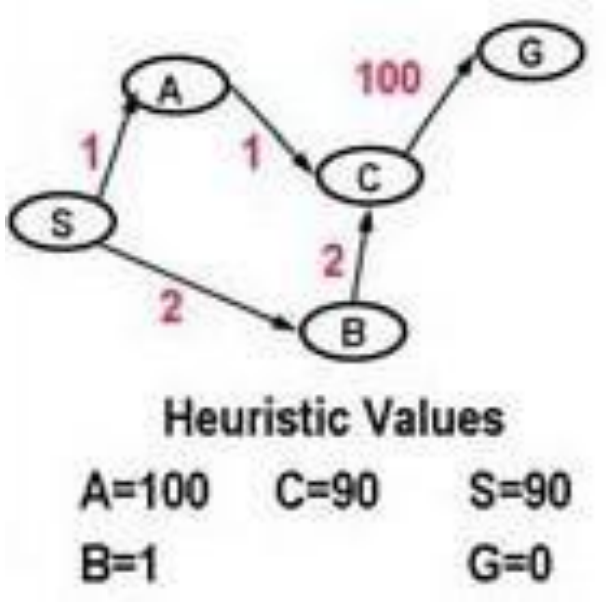
- Pick the best element of Q
- Add path extension to Q



	Q
	(90 S)
	(101 A S) (3 B S)
	(94 C B S) (101 A S)
	(101 A S) (104 G C B S)
	(92 C A S) (104 G C B S)
	(102 G C A S) (104 G C B S)

A* with Expanded List

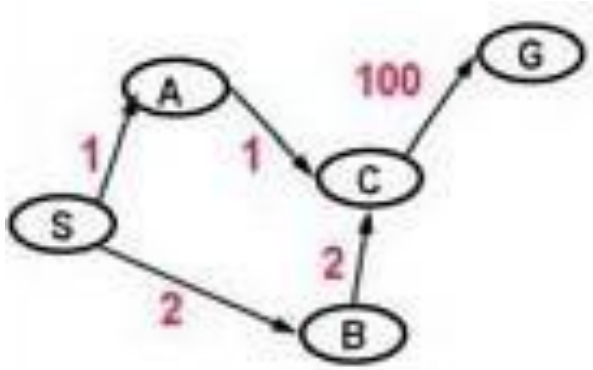
- Pick the best element of Q
- Add path extension to Q



	Q	Expanded
	(90 S)	S
	(101 A S) (3 B S)	S, B
	(94 C B S) (101 A S)	S, B C
	(101 A S) (104 G C B S)	S, B,C, A
	~~(92 C A S)~~ (104 G C B S)	S, B, C, A, G

A* with expanded List

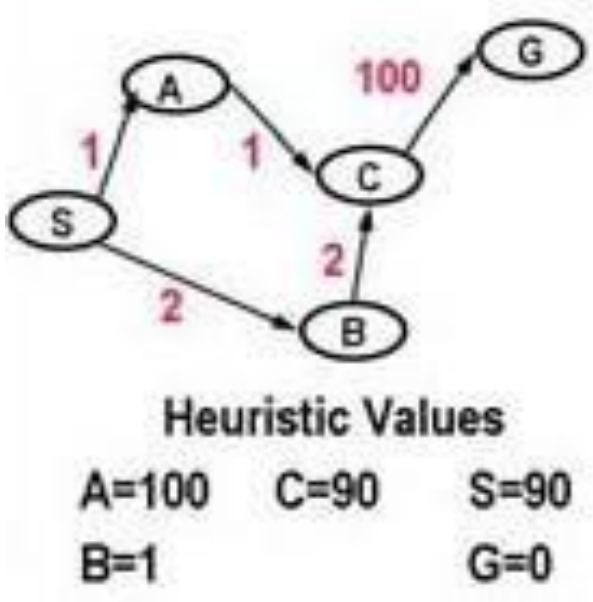
- Find the consistency of the all node



Consistency of S **S to A** **S to B**	**Consistency of A** **A to B**	**Consistency of B** **B to C**
$h(S) - h(\mathrm{A}) \leq c(S, a, A)$ $90 - 100 \leq 1$ $-10 \leq 1$	$h(A) - h(\mathrm{C}) \leq c(A, a, C)$ $100 - 90 \leq 1$ $10 \leq 1$	$h(B) - h(\mathrm{C}) \leq c(B, a, C)$ $1 - 90 \leq 2$ $-89 \leq 2$
$h(S) - h(\mathrm{B}) \leq c(S, a, A)$ $90 - 1 \leq 1$ $89 \leq 1$		

A* with expanded List

- Find the consistency of the all node



Consistency of S S to A S to B	Consistency of A A to B	Consistency of B B to C
$h(S) - h(\mathrm{A}) \leq c(S, a, A)$ $90 - 100 \leq 1$ $-10 \leq 1$	$h(A) - h(\mathrm{C}) \leq c(A, a, C)$ $100 - 90 \leq 1$ $10 \leq 1$	$h(B) - h(\mathrm{C}) \leq c(B, a, C)$ $1 - 90 \leq 2$ $-89 \leq 2$
$h(S) - h(\mathrm{B}) \leq c(S, a, B)$ $90 - 1 \leq 2$ $89 \leq 2$		

Consistency

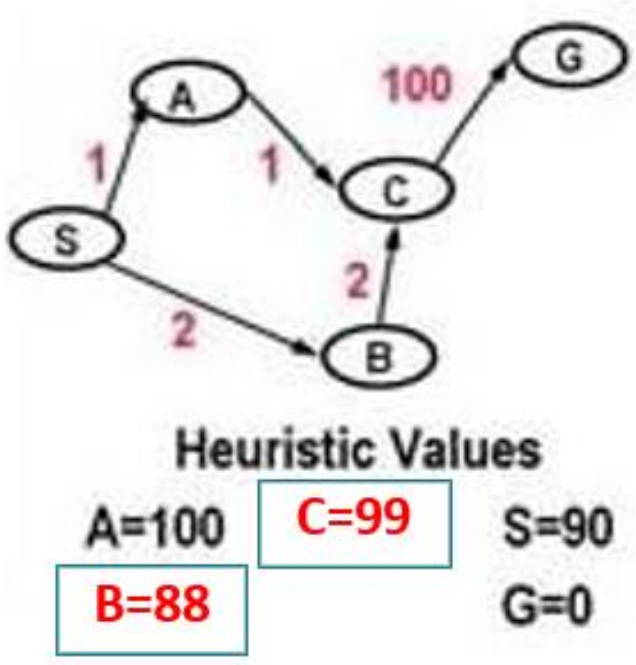
$$h(S) - h(\mathrm{A}) \leq c(S, a, A)$$
$$h(S) \leq c(S, a, A) + h(A)$$

$h(S) - h(\mathrm{B}) \leq c(S, a, B)$ $90 - 1 \leq 2$ $89 \leq 2$
$h(S) - c(S, a, B) \leq h(B)$ $h(B) \leq 90 - 2$ $h(B) \leq 88$

$h(A) - h(\mathrm{C}) \leq c(A, a, C)$ $100 - 90 \leq 1$ $10 \leq 1$
$h(A) - c(A, a, C) \leq h(C)$ $h(C) \leq 100 - 1$ $h(B) \leq 99$

Updated Heuristics

	Q	Expanded
1	(90 S)	S
2	(101 A S) (90 B S)	S, B
3	(103 C B S) (101 A S)	S, B, A
4	(101 C A S) ~~(103 C B S)~~	S, B, A, C,
5	(102 G C A S)	S, B, A, C, G



Recursive Best First Sarch

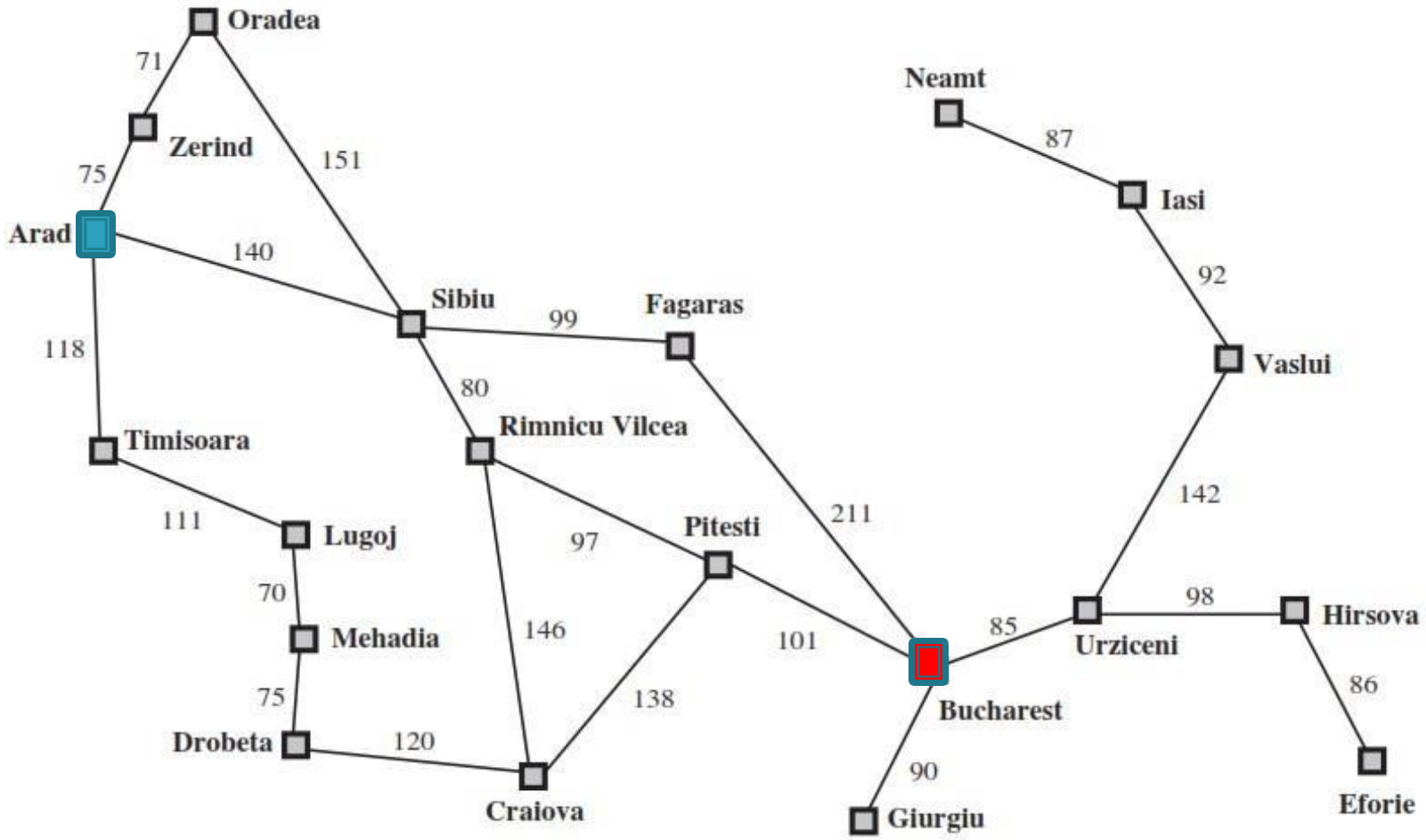
- Recursive Best-First Search (RBFS) is a simple recursive algorithm that attempts to mimic the operation of standard best-first search and A* search, but using only linear space
- It uses the $\boldsymbol{f\_limit}$ variable to keep track of the $\boldsymbol{f\_value}$ of the best alternative path.
- If the current node exceeds this limit, the recursion unwinds back to the alternative path.
- As the recursion unwinds, RBFS replaces the $\boldsymbol{f\_value}$ of each
- node along the path with the best $\boldsymbol{f\_value}$ of its children.

Recursive Best First Search

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution, or failure
    return RBFS(problem, MAKE-NODE(problem.INITIAL-STATE), ∞)

function RBFS(problem, node, f_limit) returns a solution, or failure and a new f-cost limit
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  successors ← [ ]
  for each action in problem.ACTIONS(node.STATE) do
      add CHILD-NODE(problem, node, action) into successors
  if successors is empty then return failure, ∞
  for each s in successors do /* update f with value from previous search, if any */
      s.f ← max(s.g + s.h, node.f))
  loop do
      best ← the lowest f-value node in successors
      if best.f > f_limit then return failure, best.f
      alternative ← the second-lowest f-value among successors
      result, best.f ← RBFS(problem, best, min(f_limit, alternative))
      if result ≠ failure then return result
```

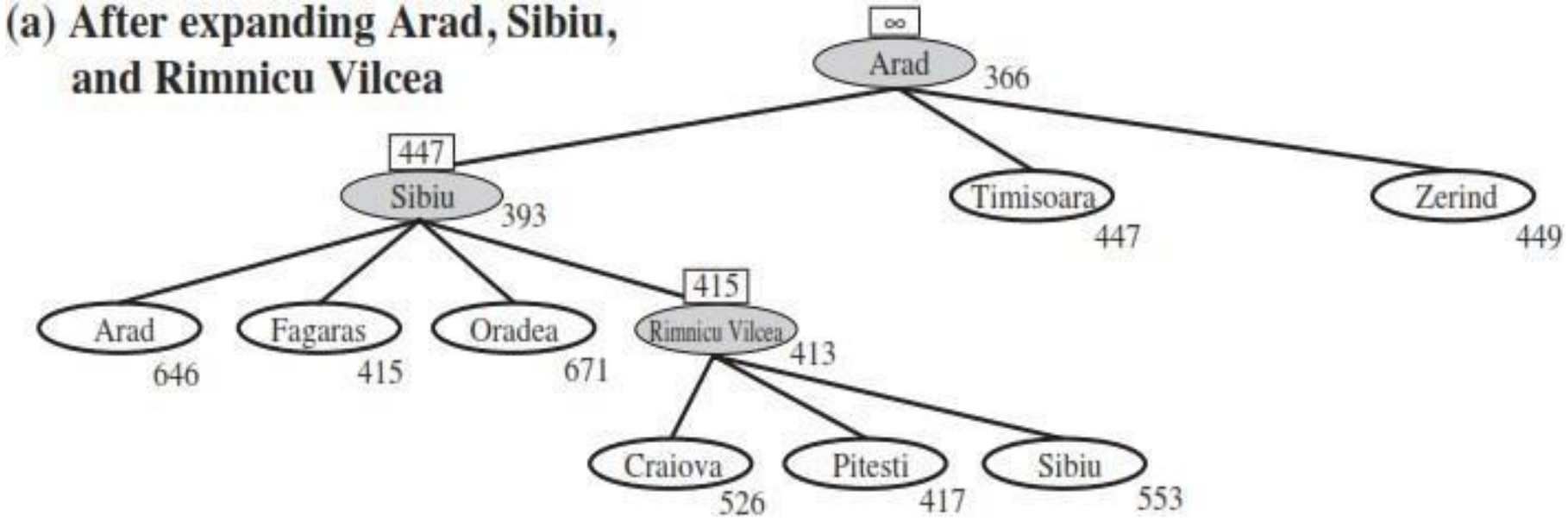
RBFS Example



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

RBFS

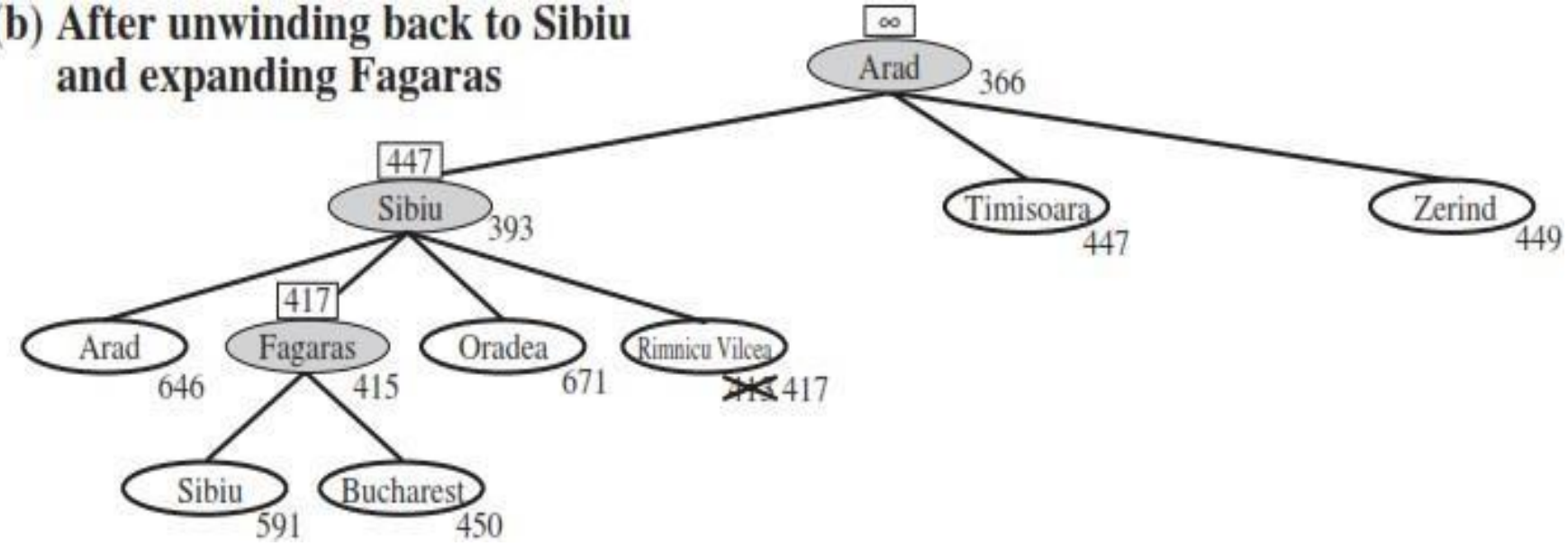
- The ***f_limit*** value for each recursive call is shown on top of each current node, and every node is labeled with its ***f_cost***.



RBFS Example

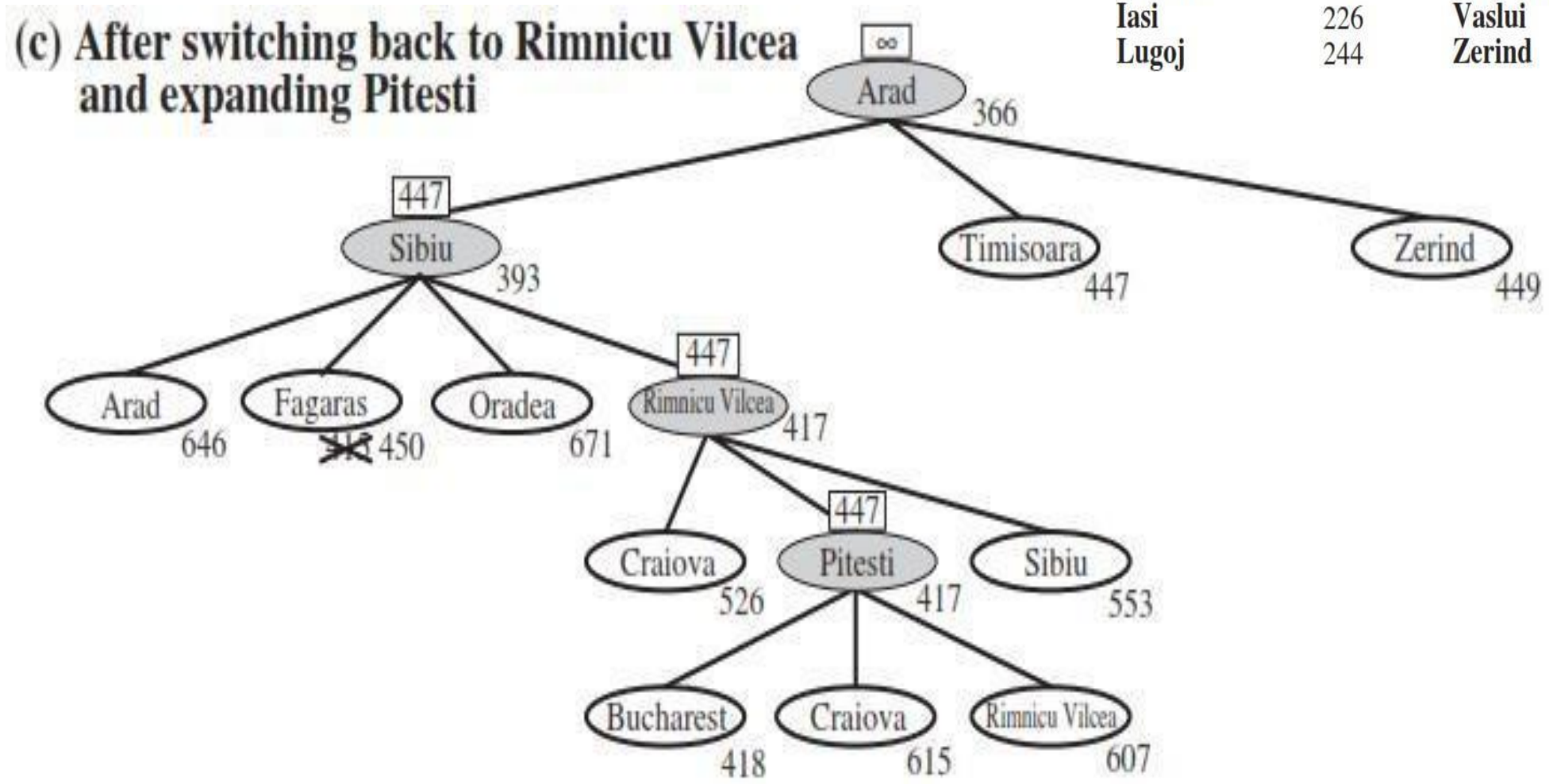
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

(b) After unwinding back to Sibiu and expanding Fagaras



RBFS Example

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

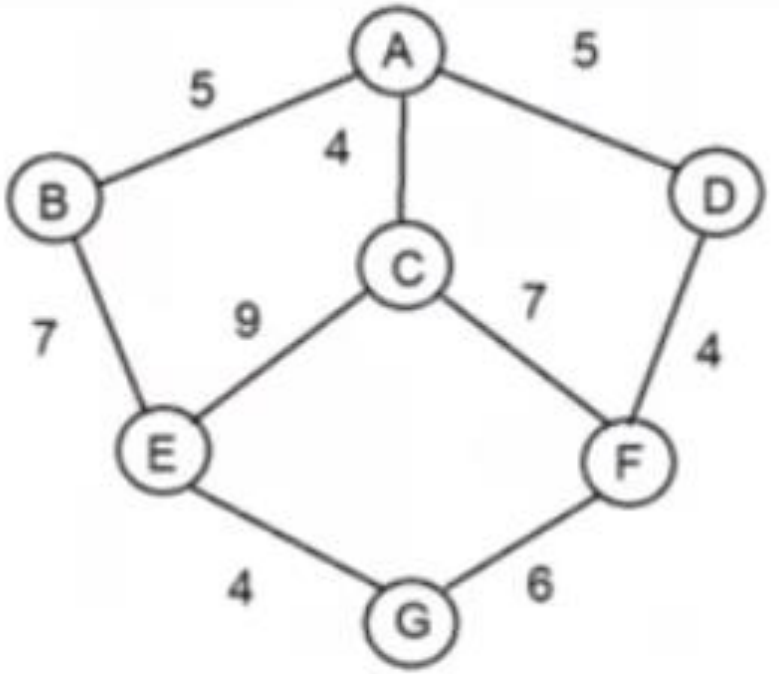


Recursive Best First Search

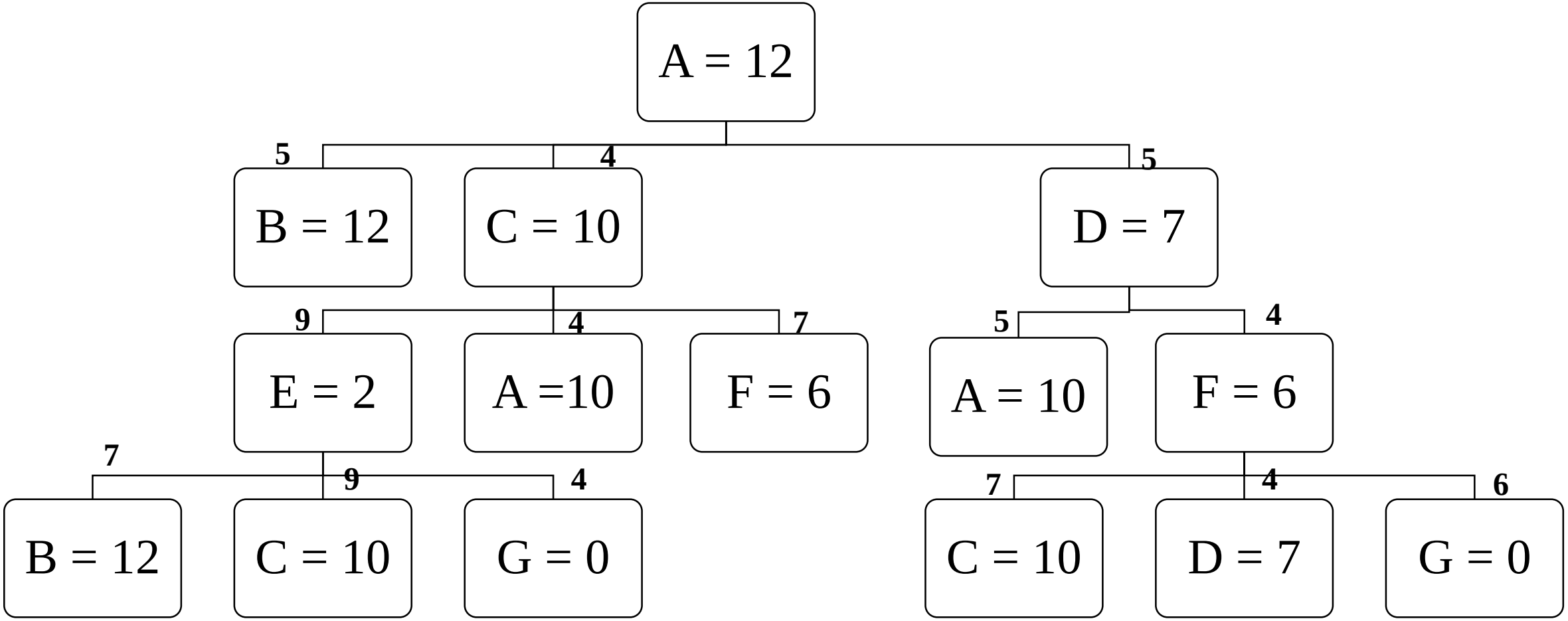
- Like A∗ tree search, RBFS is an **optimal algorithm** if the *heuristic function $h(n)$ is admissible.*
- Its **space complexity** is *linear* in the depth of the deepest optimal solution,
- Its **time complexity** is rather difficult to characterize: it depends both on
  - The *accuracy of the heuristic function* and
  - *How often the best path changes* as nodes are expanded.

RBFS Example:

node	Heursitics
A	12
B	12
C	10
D	7
E	2
F	6
G	0

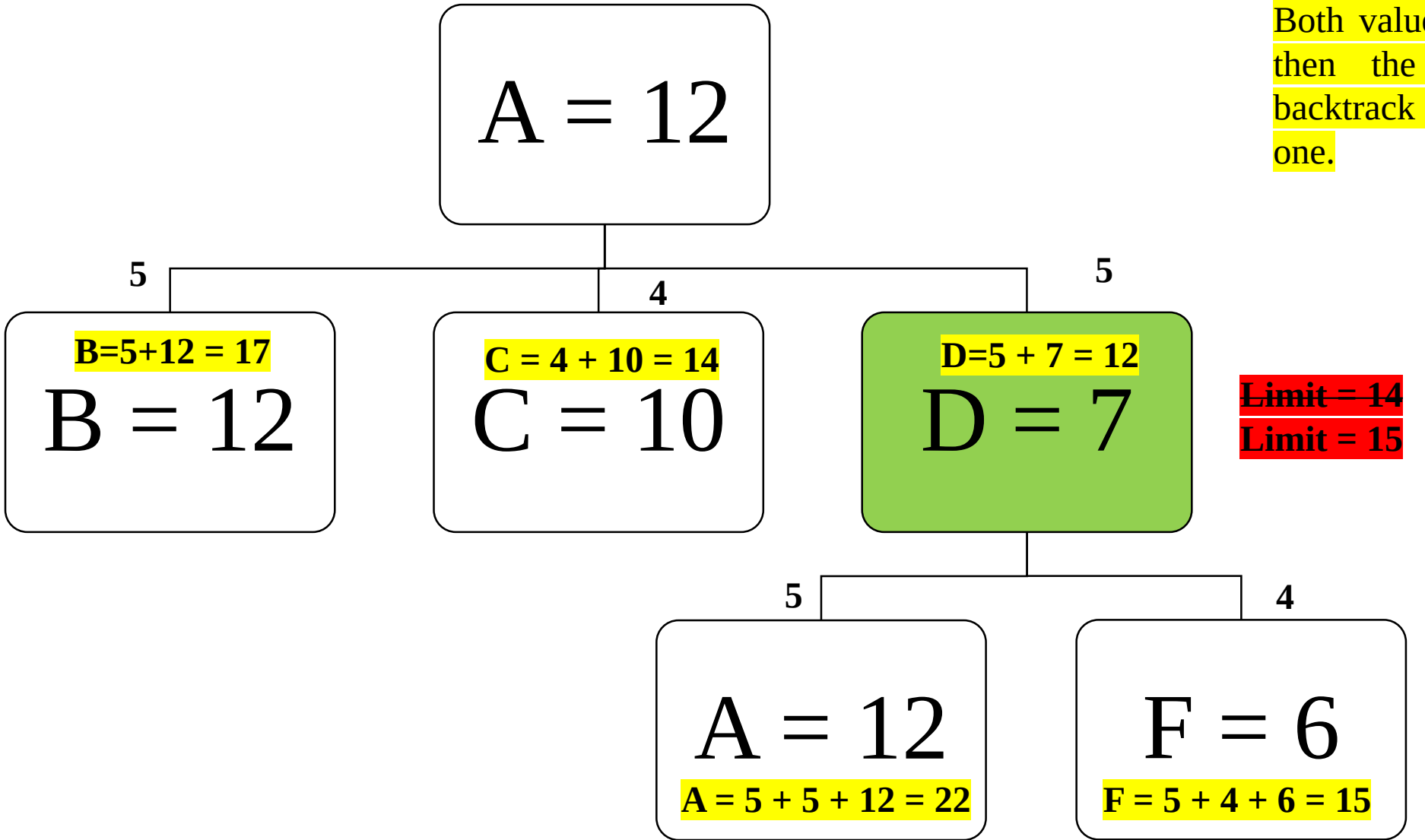


Recursive Best First Search



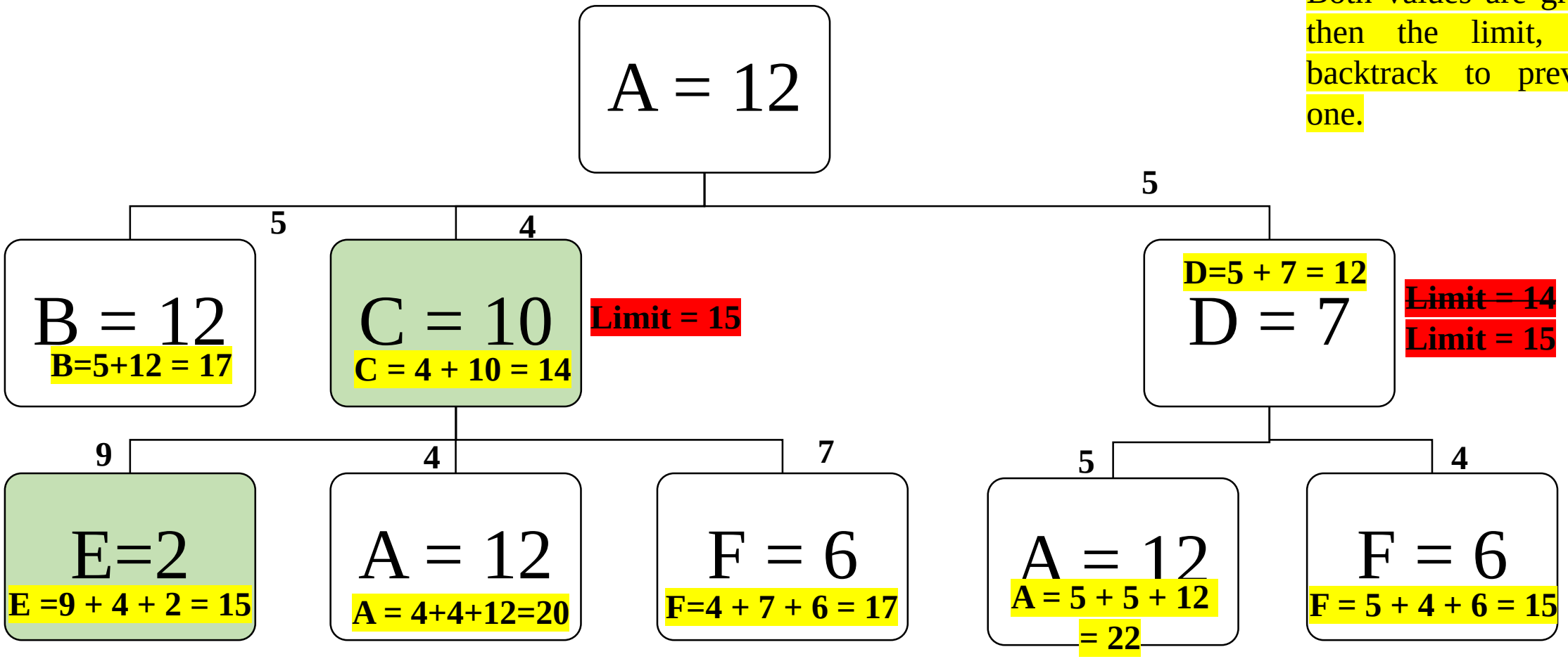
Recursive Best First Search

Limit = 14
Cost of F = 15
Cost of = 22
Both values are greater then the limit, now backtrack to previous one.



Recursive Best First Search

Limit = 15
Cost of E = 15
Cost of A= 20
Cost of F = 17
Both values are greater then the limit, now backtrack to previous one.



Recursive Best First Search